Imp Language Extension in Haskell

Calinov Cosmin

March 12, 2026

1 Introduction

This project implements an interpreter for an extended version of the IMP toy language using Haskell. The goal is to define the language formally (Syntax and Semantics) and verify its properties using property-based testing (QuickCheck).

The language builds upon the standard IMP imperative language by adding:

- **Rich Arithmetic:** Support for Modulo (`%`) and Power (`^`) operators.
- **Boolean Logic:** Full boolean expression support including comparison and logical operators.
- **Scoping:** A `Block` construct to group statements.
- **Concurrency:** A `Par` construct for the parallel execution of statements, which merges the resulting states deterministically.

The interpreter relies on Big-Step Operational Semantics (Natural Semantics) to define the execution rules.

2 Abstract Syntax Tree

2.1 Formal Grammar

What programs look like

⟨*Exp*⟩ ::= ⟨*Integer*⟩ | ⟨*Variable*⟩
| ⟨*Exp*⟩ '`+`' ⟨*Exp*⟩ | ⟨*Exp*⟩ '`*`' ⟨*Exp*⟩
| ⟨*Exp*⟩ '`-`' ⟨*Exp*⟩ | ⟨*Exp*⟩ '`/`' ⟨*Exp*⟩
| ⟨*Exp*⟩ '`%`' ⟨*Exp*⟩ | ⟨*Exp*⟩ '`^`' ⟨*Exp*⟩
| '`true`' | '`false`'
| ⟨*Exp*⟩ '`==`' ⟨*Exp*⟩ | ⟨*Exp*⟩ '`<=`' ⟨*Exp*⟩
| ⟨*Exp*⟩ '`>=`' ⟨*Exp*⟩ | ⟨*Exp*⟩ '`<`' ⟨*Exp*⟩
| ⟨*Exp*⟩ '`>`' ⟨*Exp*⟩ | '`not`' ⟨*Exp*⟩
| ⟨*Exp*⟩ '`&&`' ⟨*Exp*⟩ | ⟨*Exp*⟩ '`||`' ⟨*Exp*⟩

⟨*StmtList*⟩ ::= ⟨*Stmt*⟩ | ⟨*Stmt*⟩ ‘`;`’ ⟨*StmtList*⟩

⟨*Stmt*⟩ ::= ‘`skip`’
| ⟨*Variable*⟩ ‘`:=`’ ⟨*Exp*⟩
| ⟨*Stmt*⟩ ‘`;`’ ⟨*Stmt*⟩
| ‘`if`’ ⟨*Exp*⟩ ‘`then`’ ⟨*Stmt*⟩ ‘`else`’ ⟨*Stmt*⟩
| ‘`while`’ ⟨*Exp*⟩ ‘`do`’ ⟨*Stmt*⟩
| ‘`{`’ ⟨*StmtList*⟩ ‘`}`’
| ‘`par`’ ‘`{`’ ⟨*StmtList*⟩ ‘`}`’

2.2 Data Types

Common Data Types

```haskell
newtype Variable = Var String deriving (Eq, Ord, Show)

data Value = VInt Int | VBool Bool deriving (Eq, Show)

newtype MyState = List [(Variable, Value)]
  deriving (Eq, Show)
```

Expression

```haskell
data Exp = EVar Variable
         | EInt Int
         | EAdd Exp Exp
         | EMul Exp Exp
         | ESub Exp Exp
         | EDiv Exp Exp
         | EMod Exp Exp
         | EPow Exp Exp
         | BExp Bool
         | BEq Exp Exp
         | BLt Exp Exp
         | BGt Exp Exp
         | BLte Exp Exp
         | BGte Exp Exp
         | BAnd Exp Exp
         | BOr Exp Exp
         | BNot Exp
deriving Show
```

Note: `EMod` and `EPow` correspond to the `%` operator and `^` operator, respectively.

Statement

```haskell
data Stmt = Assign Variable Exp
          | Seq Stmt Stmt
          | If Exp Stmt Stmt
          | While Exp Stmt
          | Skip
          | Block [Stmt]
          | Par [Stmt]
deriving Show
```

3 State

The state in the extended language is just a function that takes a Variable and returns its assigned Value.

$$\sigma : Var \to Val$$

4 Formal Semantics

What programs do.

4.1 Skip

$$\frac{}{\langle \text{skip}, \sigma \rangle \Downarrow \sigma}$$

4.2 Evaluation of Expressions

The evaluation of expressions relies on the state σ to retrieve variable values.

Constants and Variables:

$$\frac{}{\langle n, \sigma \rangle \Downarrow n} \quad (n \in \mathbb{Z}) \qquad \frac{}{\langle b, \sigma \rangle \Downarrow b} \quad (b \in \{\text{true}, \text{false}\}) \qquad \frac{x \in \text{Dom}(\sigma)}{\langle x, \sigma \rangle \Downarrow \sigma(x)}$$

Binary Operations: For any binary operator $\oplus$ (e.g., $+$, $-$, $*$, $\leq$, $\wedge$):

$$\frac{\langle e_1, \sigma \rangle \Downarrow v_1 \quad \langle e_2, \sigma \rangle \Downarrow v_2}{\langle e_1 \oplus e_2, \sigma \rangle \Downarrow v_1 \oplus_{val} v_2}$$

4.3 Assignment

$$\frac{\langle e, \sigma \rangle \Downarrow v}{\langle x := e, \sigma \rangle \Downarrow \sigma[x \mapsto v]}$$

4.4 Sequencing

$$\frac{\langle S_1, \sigma \rangle \Downarrow \sigma' \quad \langle S_2, \sigma' \rangle \Downarrow \sigma''}{\langle S_1; S_2, \sigma \rangle \Downarrow \sigma''}$$

4.5 Conditional (If)

True Branch

$$\frac{\langle b, \sigma\rangle \Downarrow \text{true} \quad \langle S_1, \sigma\rangle \Downarrow \sigma'}{\langle \text{if } b \text{ then } S_1 \text{ else } S_2, \sigma\rangle \Downarrow \sigma'}$$

False Branch

$$\frac{\langle b, \sigma\rangle \Downarrow \text{false} \quad \langle S_2, \sigma\rangle \Downarrow \sigma'}{\langle \text{if } b \text{ then } S_1 \text{ else } S_2, \sigma\rangle \Downarrow \sigma'}$$

4.6 While

True

$$\frac{\langle b, \sigma\rangle \Downarrow \text{true} \quad \langle S, \sigma\rangle \Downarrow \sigma' \quad \langle \text{while } b \text{ do } S, \sigma'\rangle \Downarrow \sigma''}{\langle \text{while } b \text{ do } S, \sigma\rangle \Downarrow \sigma''}$$

False

$$\frac{\langle b, \sigma\rangle \Downarrow \text{false}}{\langle \text{while } b \text{ do } S, \sigma\rangle \Downarrow \sigma}$$

4.7 Block

A block of statements behaves like a recursive sequence.

$$\frac{}{\langle \text{block } [], \sigma\rangle \Downarrow \sigma} \qquad \frac{\langle S, \sigma\rangle \Downarrow \sigma' \quad \langle \text{block } Ss, \sigma'\rangle \Downarrow \sigma''}{\langle \text{block } (S : Ss), \sigma\rangle \Downarrow \sigma''}$$

4.8 Parallel Execution (Par)

Parallel execution evaluates a list of statements $[S_1, \ldots, S_n]$ concurrently, all starting from the initial state σ. The final state is obtained by merging the resulting states.

$$\frac{\forall i \in \{1..n\}, \langle S_i, \sigma\rangle \Downarrow \sigma_i \quad \sigma' = \text{merge}(\sigma_n, \ldots, \sigma_1)}{\langle \textbf{par } [S_1, \ldots, S_n], \sigma\rangle \Downarrow \sigma'}$$

Where *merge* resolves conflicts by prioritizing the last statement (Last Writer Wins):

$$\text{merge}(\sigma_n, \ldots, \sigma_1)(x) = \begin{cases} \sigma_n(x) & \text{if } x \in \text{Dom}(\sigma_n) \\ \sigma_{n-1}(x) & \text{if } x \notin \text{Dom}(\sigma_n) \text{ and } x \in \text{Dom}(\sigma_{n-1}) \\ \ldots & \\ \sigma_1(x) & \text{otherwise} \end{cases}$$

5 Property-based Testing using QuickCheck

5.1 About QuickCheck

QuickCheck is library for property-based testing. Unlike traditional unit testing, where specific inputs are manually crafted, QuickCheck allows programmers to define general *properties* that functions must satisfy. These properties are formulated as Haskell functions that take randomly generated inputs and return a boolean indicating success or failure.

To use QuickCheck with custom data types, the `Arbitrary` typeclass must be implemented. This tells the library how to generate random values for a specific type. In this project, `Arbitrary` instances were created for `Exp` and `Stmt` to randomly generate syntax trees.

The `sized` function is used to control the size of these generated structures, preventing infinite recursion. Additionally, the `frequency` function guides the random generation by assigning probabilities to different constructors. For instance, binary constructors (like `EAdd` or `Seq`) are given a higher frequency and split the size parameter in half for recursive calls to ensure a balanced distribution, while leaf nodes decrement the size linearly.

5.2 Skip doesn't modify the state of the program

Skip statement's usefulness is shown in this property.

$$\text{stmt}(\sigma, \text{Skip}) = \sigma$$

5.3 Sequential execution is compositional

A staple for all programming languages that are deterministic.

$$\text{stmt}(\sigma, \text{Seq } s_1 s_2) = \text{stmt}(\text{stmt}(\sigma, s_1), s_2)$$

5.4 Assigning a variable and then getting its value returns the assigned value

Basic testing for state's functionality.

$$\text{get}(\text{stmt}(\sigma, \text{Assign } x\ v), x) = v$$

5.5 Sequencing statements is associative

Doesn't matter in which order you run the operation, the end result is the same.

$$\text{stmt}(\sigma, \text{Seq}(\text{Seq}(s_1, s_2), s_3) = \text{stmt}(\sigma, \text{Seq}(s_1, \text{Seq}(s_2, s_3))$$

5.6 Program execution is deterministic

This property illustrates that the language's evaluation rules are consistent: running the same statement twice from the same initial state yields the exact same final state.

$$\text{stmt}(\sigma, s) = \text{stmt}(\sigma, s)$$

Testing Challenges: A major challenge in testing this property was the `While` construct. Randomly generated `While` loops frequently result in non-terminating programs, causing the test suite to hang.

We considered several solutions:

- **Excluding While loops:** Removing `While` from the `Arbitrary` instance avoids non-termination but leaves a critical part of the language untested.
- **Fuel Parameter in Semantics:** Adding a "fuel" argument to the interpreter (e.g., `stmt :: Int -> State -> Stmt -> State`) to forcibly stop execution. This was rejected because it would alter the language's semantics and signature purely for testing purposes.

Adopted Solution - Bounded Generation: We implemented a method called **Bounded Generation** within the `Arbitrary` instance. Instead of generating raw `While` loops, we generate "safe" loops that include a built-in termination mechanism.

The generator constructs a loop where:

1. A fresh variable (e.g., `__limit`) is initialized with a small random integer (the fuel).
2. The loop condition is augmented to check if `__limit > 0`.
3. The loop body is modified to decrement `__limit` at each step.

This approach ensures that all generated programs terminate by construction, allowing QuickCheck to verify the determinism property without modifying the interpreter itself.

6 Usage Examples

6.1 Example 1: Loop and Arithmetic

```
main :: IO ()
main = do
  let x = Var "x"
  y = Var "y"
  example = Seq
    (Assign x (EInt 0))
        (While (BLt (EVar x) (EInt 3)) (Block
          [ Assign y (EAdd (EVar x) (EInt 1))
          , Assign x (EAdd (EVar x) (EInt 1))
          ]))

    final = stmt (List []) example
    (val, _) = get final y

  putStrLn $ "Result of program (value of y): " ++ show val
```

Output: Result of program (value of y): VInt 3

6.2 Example 2: Parallel Assignment

```
stmt5 = Par
  [ Assign (Var "x") (EInt 1)
  , Assign (Var "y") (EInt 2)]
  finalState3 = stmt (List []) stmt5
  valX = fst (get finalState3 (Var "x"))
  valY = fst (get finalState3 (Var "y"))
```

Result: `valX` is `VInt 1`, `valY` is `VInt 2`.

7 Conclusion

This project successfully extended the IMP language with arithmetic operators, local scoping, and deterministic parallelism. The implementation was verified using the QuickCheck library, ensuring adherence to the formal semantics defined using Big-Step operational rules. The use of property-based testing proved effective in identifying edge cases, particularly in non-terminating loops and state consistency during parallel execution.

8 References

- MIT Press Direct. *The Formal Semantics of Programming Languages: An Introduction* `https://direct.mit.edu/books/monograph/4338/`

- Chalmers University of Technology. *QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs* `https://dl.acm.org/doi/epdf/10.1145/351240.351266`
- Hackage. *QuickCheck: Automatic testing of Haskell programs* `https://hackage.haskell.org/package/QuickCheck`
- sulzmann.github.io. *QuickCheck: Automatic Testing* `https://sulzmann.github.io/ProgrammingParadigms/lec-haskell-testing.html`
- University of Nottingham. *Programming language semantics: It's easy as 1,2,3* `https://people.cs.nott.ac.uk/pszgmh/123.pdf)`